
MP7: Postprocessing with Multiple Camera Passes

Due Time: Refer to the course web-site

Objectives

Verify that we understand and are able to work with:

- Unity specific skills:
 - Setup and work with post-processing passes
- General coding skills/concepts: given a mathematic expression, able to
 - identify and compute values based on 3D scene elements
 - implement the expression in a shader
- Shader and relevant general concepts:
 - Work with multiple camera passes and non-Color buffer results (general Render Texture)
 - Implement multi-pass postprocessing algorithm
 - Simulate fake fog and focus effects as postprocessing passes

Approach

Based on our MP5 (or MP6) solution, we will define GroundFog and Torch shaders to integrate ground fog, volumetric, and torch effects.

For this assignment, we will assume (do you know why these assumptions):

- There is a very large ground plane located at $y=0$, so large that we cannot see the boundaries
- We always:
 - Examine the scene from above the ground plane, and,
 - Examining volumetric medium from the outside, that, we will never look outwards from inside of the volume

Fog modeling.

Recall that we modeled the transmittance, or, how quickly light gets attenuated when traveling through participating media as,

$$T = e^{-\tau}$$

Where,

$$\tau = \sigma D$$

In this formulation,

σ – Extinction Coefficient A parameter set by the
 D – Visible interval of media Computed from viewing ray

We model the color of the participating media as,

$$\text{computedFogColor} = \text{FogColor} \times \text{FogScattering} \times (1 - T)$$

Where, FogColor is a color and FogScattering is a float, both being parameters that can be set by the user. In this programming assignment, to further control the rate of drop off, you will modify the above given τ by computing,

$$\tau' = \text{smoothstep}(0, 1, \text{pow}(\tau, \text{dropOff}))$$

Where, dropOff is a float parameter supplied by the user. In our model, the final color for the pixel is:

$$\text{finalPixelColor} = T \times \text{ObjectColor} + \text{computedFogColor}$$

Background Math.

If we have a viewing ray with origin P , and a viewing direction, V , looking downwards at a ground plane at $y = h$, the ray would intersect the ground plane at:

$$P + tV = (x, h, z)$$

Where, (x, z) is the visible point on the plane. Note, since the ray is a straight line, it is true that,

$$P.y + tV.y = h$$

Or,

$$t = \frac{h - P.y}{V.y}$$

With this formulation, we can now find t when h is a given height and when $h = 0$ (the ground plane). This means, we can now compute a world coordinate solution of fog above the ground for a very large ground plane, if we only allow the user to look from above downwards towards the ground.

GroundFog shader.

You will implement barebone ground fog shader based on the above formulation for a fog that covers the ground of the entire scene at a height, h , and in this case,

$$D = \frac{\text{visible interval length}}{h}$$

Your shader must allow the user to provide the parameters for:

- *X/Y min/max dimensions of the world ground plane. You can use these parameters for debugging your volume interval computation*
- *FogColor*
- *FogHeight*
- *Fog extinction*
- *Fog drop off*
- *Fog scattering*

Additionally, you must allow the user to examine the computed results of each of the following parameters:

- *Tau (τ')*
- *Intersection Interval (D)*
- *Transmittance (T)*
- *Computed Fog Color*

Torch shader.

I refer to this as torch shader because it sort of behaves like one, and with volumetric fog acting (badly) as the flame around the torch or something. Instead of creating blurring/focusing effect based on the distance from the camera, what if we can blur and dim specific parts of the world? The alternative description is, what if we can allow focusing on specific region of the scene and blur/dim the rest of the rendered result? Based on this thinking, here is the specification of the torch shader:

- *The torch: TorchPosition, TorchRadius, and TorchColor: Describes the position, the illuminating radius, and the color of the torch. In my implementation, the TorchColor does **not** affect the world, it acts as the color for the volumetric effect around the torch. The darkening of the world is a result of the FarIntensity to be discussed next.*

- *The illumination: FarDistance and FarIntensity. FarDistance is where the torch stops illuminate and all objects farther than FarDistance are blurred by 15x15 box filter and dimmed by the FarIntensity. For regions between FogRadius and FarDistance, we borrow from the distance attenuation formulation:*
 - *d: distance between a visible position and the TorchPosition*
 - *if $d < \text{TorchRadius}$: show the existing color of the visible point*
 - *otherwise:*
 - *$n: d - \text{TorchRadius}$*
 - *$f: \text{FarDistance} - \text{TorchRadius}$*
 - *blend: $\text{smoothstep}(0.0, 1.0, 1.0 - (n * n) / (f * f))$;*
 - *Objects that are in between TorchRadius and FarDistance are linear blended based between original color and blurred far color on blend.*
- *Torch volumetric control: TorchExtinction, TorchDropoff, TorchScattering*
 - *For visible volumetric interval, define:*

$$D = \frac{\text{visible interval length}}{0.5 \times \text{TorchRadius}}$$
- *Torch Visual Options: to verify your understanding and cleanness of implementation, you will support*
 - *ShowVolumetric: switching the volumetric fog on/off*
 - *ShowBlur: switching the blurring effect on/off*
 - *ReverseBlur: instead of blurring and darkening the far, blur and darken inside the TorchRadius*

To facilitate your debugging and my grading, you will support examining the individual computed parameters including:

- *Regions: separate the regions into within the TorchRadius (in Green), the transition region (display the blending factor), and regions that will be complete blurred (in Red)*
- *Tau (τ')*
- *Intersection Interval (D)*
- *Transmittance (T)*
- *Computed Torch Color*

For convenience of testing, please make sure in your initial scene one object is clearly in focus with the rest of the scene clearly blurred and dimmed.

Hints:

- *Ground Fog is trivial, or, impossible, depending on if you understand the intersection math. I would*
 - *understand the math, then,*
 - *formulate in shading HLSL, then,*
 - *test with visualizing the intersection results in some constant color, and, lastly,*
 - *implement the volumetric model*
 - *Here is my [fog control script](#)*
- *Torch: I think the trickiest part (for me) was the difference between distance to the torch center (how blur) and distance from the eye (how much fog). I had the two mixed up and almost ended my life.*
 - *Work on the different visuals (on/off of fog/blur) very last, make sure everything is working, save a copy, and then, think about how to turn the effects on/off, or flip the blurring results.*
 - *MAKE SURE to save a copy before you start to re-organize*
 - *Here is my [torch control script](#)*

Post processing interface. *Very last thing, we want to examine each of these two shaders individually. Additionally, although it does not make much sense as the spec did not define the details, let's support processing the two shaders in subsequent order one after the other in a user choosing order. In other words,*

support four different post-processing order: Fog only, Torch only, Fog first then Torch, and, Torch first then Fog. Notice, the last two produces results that are different.

Credit Distribution

		Credits
1	Unity Scene Setting 10	
	At least 6 objects, with large ground plane at $y=0$	2
	All objects are Phong illuminated	
	At least 2 directional light with on/off	4
	Initial scene setting is reasonable	4
	Proper lighting, no over-saturation of color	
2	Ground Fog: 26	
	X/Y range of ground fog coverage	6
	Height as WC y	8
	Color	2
	Extinction Coefficient	2
	Drop off	2
	Scattering	2
	Debug: : Tau, Interval, T, Computed Color	4
	Initial setting: objects immersed partially in fog	2
3	Torch Shader 39	
	Position + Radius + Color	10
	Far distance + Intensity	5
	Proper transition	6
	Volumetric: Extinction, DropOff, Scattering	6
	Visual mode: Fog on/off, Blur on/off, Invert	8
	Debug: Region, Tau, Interval, T, Computed Color	4
	Initial setting: only one object clearly in focus	2
4	UI and Control 16	
	Torch Control located on SceneControl object	2
	Fog and Torch controls integrated on SceneControl	4
	Post Process control integrated with SceneControl	10
	Toggle effects on/off, control sequence	
6	Submission	
	Proper folders/unity version	5
	Total	100

This assignment counts 10% towards your final grade for this class.